PEMROGRAMAN VISUAL

Pertemuan 5

Struktur Percabangan If–Else

Membuat aplikasi visual yang dapat mengambil keputusan berdasarkan kondisi

Insidini Fawwaz, M.Kom

Mengapa Program Membutuhkan Percabangan?

Tanpa percabangan

Program selalu menjalankan instruksi yang sama, apa pun kondisi inputnya.

Dengan percabangan

Program memeriksa kondisi terlebih dahulu, lalu memilih tindakan yang sesuai.

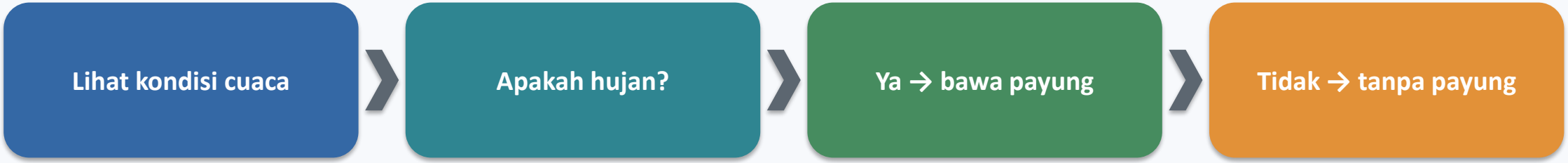
Contoh

Jika nilai ≥ 60 → Lulus.
Jika tidak → Tidak Lulus.

KONSEP

Analogi Percabangan dalam Kehidupan

Percabangan = memilih tindakan berdasarkan suatu kondisi.



Inti konsep

Program juga melakukan hal yang sama: mengevaluasi kondisi yang bernilai True atau False, kemudian menentukan blok kode yang dijalankan.

Konsep Dasar If–Else

- Program akan **mengecek suatu kondisi terlebih dahulu**.
- Hasil kondisi tersebut hanya ada dua: True (benar) atau False (salah).

```
If nilai >= 75 Then
    MessageBox.Show("Lulus")
Else
    MessageBox.Show("Tidak
Lulus")
End If
```

IF

Memeriksa kondisi.

ELSE

Alternatif ketika kondisi tidak terpenuhi.

END IF

Menandai akhir struktur percabangan.

Bentuk 1 — If...Then

Dipakai jika kita **hanya ingin melakukan sesuatu ketika kondisi benar**.
Kalau kondisi salah, tidak melakukan apa-apa.

```
If kondisi Then
    perintah
End If


Contoh:
If umur >= 17 Then
    lblStatus.Text = "Boleh membuat
SIM"
End If
```

Kapan digunakan?

Ketika program tidak perlu melakukan tindakan khusus saat kondisi bernilai False.

Contoh If...Then pada Form

Klik tombol untuk memeriksa jumlah pembelian.

Komponen Form

TextBox: txtJumlah
Button: btnCek
Label: lblInfo

```
Private Sub btnCek_Click(...) Handles
btnCek.Click
    Dim jumlah As Integer =
CInt(txtJumlah.Text)

    If jumlah >= 5 Then
        lblInfo.Text = "Dapat diskon 10%"
    End If
End Sub
```

- **Yang membedakannya dengan If...Else**. Karena kode ini **tidak memiliki Else**, ketika syarat tidak terpenuhi program tidak memberikan aksi alternatif.

- Dim jumlah As Integer → deklarasi variabel jumlah bertipe Integer.
- txtJumlah.Text → mengambil nilai yang dimasukkan pengguna dari TextBox.
- CInt(...) → mengubah input tersebut menjadi Integer.

Bentuk 2 — If...Then...Else

Digunakan ketika ada dua kemungkinan hasil.

```
If kondisi Then
    perintahJikaBenar
Else
    perintahJikaSalah
End If
```

1. If kondisi Then
Program **memeriksa suatu kondisi**. Kondisi tersebut akan menghasilkan True atau False.
2. perintahJikaBenar
Jika kondisi **True (benar)**, bagian ini yang dijalankan.
3. Else
Artinya **“jika tidak”**. Jadi Else **tidak memiliki kondisi baru**. Ia dijalankan jika kondisi pada If tadi bernilai False.
4. perintahJikaSalah
Dijalankan ketika kondisi pada If **False (salah)**.
5. End If
Menandai bahwa percabangan selesai.

Contoh lain— If...Then...Else

- Digunakan ketika ada dua kemungkinan hasil.
- Satu kondisi, dua kemungkinan output.

```
Dim nilai As Integer =
CInt(txtNilai.Text)

If nilai >= 60 Then
    lblHasil.Text = "Lulus"
Else
    lblHasil.Text = "Tidak Lulus"
End If
```

Input

nilai

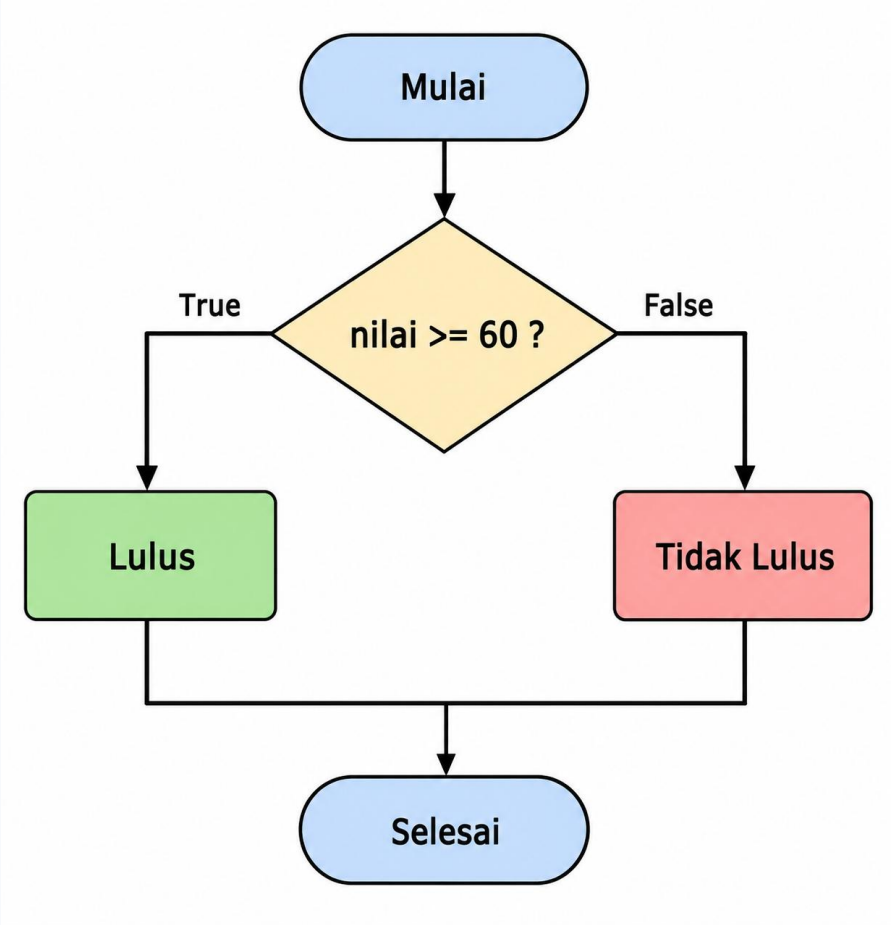
Kondisi

nilai >= 60

Output

Lulus / Tidak Lulus

Alur — If...Then...Else



Operator Perbandingan dalam Kondisi

=	sama dengan
<>	tidak sama dengan
>	lebih besar
<	lebih kecil
>=	lebih besar atau sama
<=	lebih kecil atau sama

Operator Logika

Digunakan ketika keputusan melibatkan lebih dari satu kondisi.

And

True hanya jika SEMUA kondisi True.

member = True And subtotal >= 150000

AND = semuanya harus terpenuhi.

Or

True jika SALAH SATU kondisi True.

hari = "Sabtu" Or hari = "Minggu"

Jadi kalau hari **Sabtu → True**.
Kalau hari **Minggu → True**.
Kalau hari **Senin → False**.

Not

Membalik nilai kondisi.
True → False
False → True

stokHabis = False
Artinya stok **tidak habis**.

Contoh Kondisi dengan And

Dua syarat harus terpenuhi secara bersamaan.

```
If isMember = True And subtotal >=
150000D Then
    diskonMember = subtotal * 0.05D
End If
```

Syarat 1

Pengguna adalah member.

Syarat 2

Subtotal minimal Rp150.000.

Hasil

Tambahan diskon 5%.

Bentuk 3 — If...ElseIf...Else

Digunakan ketika pilihan hasil lebih dari dua.

```
If kondisi1 Then
    perintah1
ElseIf kondisi2 Then
    perintah2
ElseIf kondisi3 Then
    perintah3
Else
    perintahLain
End If
```

Prinsip

Kondisi diperiksa dari atas ke bawah. Ketika satu kondisi True, cabang berikutnya tidak diperiksa lagi.

Contoh: Kategori Nilai

```
If nilai >= 85 Then
    grade = "A"
ElseIf nilai >= 75 Then
    grade = "B"
ElseIf nilai >= 65 Then
    grade = "C"
ElseIf nilai >= 50 Then
    grade = "D"
Else
    grade = "E"
End If
```

ElseIf artinya **“jika kondisi sebelumnya tidak terpenuhi, periksa kondisi berikutnya.”**

PENTING

Urutan Kondisi: Benar vs Salah

ElseIf berhenti pada kondisi pertama yang bernilai True.

Benar

```
If nilai >= 85 Then
  A
ElseIf nilai >= 75 Then
  B
ElseIf nilai >= 65 Then
  C
```

Bermasalah

```
If nilai >= 65 Then
  C
ElseIf nilai >= 75 Then
  B
ElseIf nilai >= 85 Then
  A
```

Nested If — Percabangan Bersarang

If dapat ditempatkan di dalam If lain atau percabangan bertingkat.

```
If nilai >= 60 Then
    If kehadiran >= 75 Then
        lblHasil.Text = "Lulus"
    Else
        lblHasil.Text = "Tidak Lulus karena
kehadiran"
    End If
Else
    lblHasil.Text = "Tidak Lulus karena
nilai"
End If
```



Gunakan seperlunya

Nested If cocok untuk keputusan bertingkat, tetapi terlalu banyak tingkat akan membuat kode sulit dibaca.

If–Else di Dalam Event

Pada aplikasi visual, percabangan biasanya dijalankan ketika event terjadi.



Pola umum

Input dari kontrol → konversi tipe data → evaluasi kondisi → proses → tampilkan output.

Contoh Form: Cek Bilangan

Kasus sederhana untuk memahami hubungan kontrol dan kode.

Desain Form

Label: Masukkan angka
TextBox: txtAngka
Button: btnCek
Label hasil: lblHasil

```
Dim angka As Integer = CInt(txtAngka.Text)

If angka Mod 2 = 0 Then
    lblHasil.Text = "Bilangan Genap"
Else
    lblHasil.Text = "Bilangan Ganjil"
End If
```

Misal:

- 8 Mod 2 = 0
- 7 Mod 2 = 1

Validasi Input Sebelum Percabangan

- TryParse artinya mencoba mengubah input. Kalau berhasil menghasilkan True dan nilainya disimpan; kalau gagal menghasilkan False tanpa membuat program langsung error.
- Input pengguna tidak selalu sesuai yang kita harapkan.

Mengapa TryParse?

Mencegah aplikasi error ketika pengguna mengetik huruf, simbol, atau mengosongkan input.

```
Dim nilai As Integer

If Integer.TryParse(txtNilai.Text, nilai)
Then
    lblHasil.Text = "Input valid"
Else
    lblHasil.Text = "Masukkan angka yang
benar"
End If
```

Ilustrasi Validasi Input dengan TryParse

Kode Program (VB)

```
Dim nilai As Integer

If Integer.TryParse(txtNilai.Text, nilai) Then
   lblHasil.Text = "Input valid"
Else
   lblHasil.Text = "Masukkan angka yang benar"
End If
```

Tampilan Form

Masukkan nilai:

80 | Cek

Hasil:

Input valid

Contoh 1: Input valid

Pengguna memasukkan angka (misal 80)

Masukkan nilai: 80 | Cek

↓

`Integer.TryParse("80", nilai)`

↓ **Berhasil (True)**

Variabel nilai berisi 80

lblHasil.Text = "Input valid"

↓

Hasil di Form

Hasil:

Input valid

Contoh 2: Input tidak valid

Pengguna memasukkan huruf (misal abc)

Masukkan nilai: abc | Cek

↓

`Integer.TryParse("abc", nilai)`

↓ **Gagal (False)**

Variabel nilai tidak berubah

lblHasil.Text = "Masukkan angka yang benar"

↓

Hasil di Form

Hasil:

Masukkan angka yang benar

Kesimpulan:

- **TryParse** mencoba mengubah input menjadi Integer.
- Jika berhasil → True dan nilai disimpan ke variabel.
- Jika gagal → False, sehingga program tidak error dan kita bisa memberikan pesan kesalahan.

Perbedaan Deklarasi, CInt, dan TryParse

1 Deklarasi Variabel

Menyiapkan tempat penyimpanan.

```
Dim nilai As Integer
```

- Membuat variabel bernama nilai dengan tipe data Integer.
- Belum ada nilai yang disimpan.

2 Inisialisasi Variabel

Memberi nilai awal.

```
Dim nilai As Integer = 80
```

- Membuat variabel nilai bertipe Integer dan langsung memberi nilai 80.
- Nilai bisa diganti nanti sesuai kebutuhan.

Inti Perbedaan

- **Deklarasi** = menyiapkan tempat penyimpanan (membuat variabel).
- **Inisialisasi** = memberi nilai awal.
- **CInt** = mengubah input menjadi tipe tertentu, **tetapi tidak aman jika input salah.**
- **TryParse** = mencoba mengubah input dan mengecek apakah berhasil. Jika gagal, hasilnya False sehingga kita bisa menangani kesalahan.

Menggunakan CInt

Mengubah input menjadi Integer (tetapi tidak aman jika input salah).

```
Dim nilai As Integer = CInt(txtNilai.Text)
```

Contoh Input Valid

Pengguna memasukkan angka.

Masukkan nilai: 123 [Proses]

CInt("123")
Berhasil diubah menjadi Integer.

nilai = 123
Program berjalan normal.

Contoh Input Tidak Valid

Pengguna memasukkan huruf.

Masukkan nilai: abc [Proses]

CInt("abc")
Terjadi error *(FormatException)*.

Program berhenti dan menampilkan pesan error.

CInt tidak mengecek input terlebih dahulu.
Jika input tidak sesuai, program bisa mengalami error (FormatException) dan berhenti.

Menggunakan TryParse

Mencoba mengubah input dan mengecek apakah berhasil.

```
Dim nilai As Integer
If Integer.TryParse(txtNilai.Text, nilai) Then
    lblHasil.Text = "Input valid"
Else
    lblHasil.Text = "Masukkan angka yang benar"
End If
```

Contoh Input Valid

Pengguna memasukkan angka.

Masukkan nilai: 123 [Proses]

Integer.TryParse("123", nilai)
Hasil: True
nilai = 123

Program berjalan normal dan bisa melanjutkan proses menggunakan nilai.

Contoh Input Tidak Valid

Pengguna memasukkan huruf.

Masukkan nilai: abc [Proses]

Integer.TryParse("abc", nilai)
Hasil: False
nilai tidak berubah

Masuk ke blok Else
Menampilkan pesan: "Masukkan angka yang benar".

TryParse lebih aman karena tidak menimbulkan error.
Program tetap berjalan dan kita bisa menangani input yang salah.

Kasus: Diskon Pemesanan Makanan

Menerapkan operator dan percabangan pada kasus yang lebih realistis.

Harga makanan = Rp35.000/porsi.

Minimal 5 porsi → diskon 10%.

Member DAN subtotal ≥ Rp150.000 → tambahan diskon 5%.

Total diskon maksimal 15%.

Setelah diskon dikenakan biaya layanan 5%.

Jika total setelah diskon < Rp200.000 → ongkir Rp10.000.

Kasus Diskon — Input, Proses, Output

Pisahkan kebutuhan program sebelum menulis kode.

INPUT

jumlahPorsi
isMember

PROSES

subtotal
diskon
totalSetelahDiskon
biayaLayanan
ongkir

OUTPUT

subtotal
diskon
totalBayar

Kasus Diskon — Percabangan 10%

Kode Program (VB)

```
Dim diskonPersen As Decimal = 0D

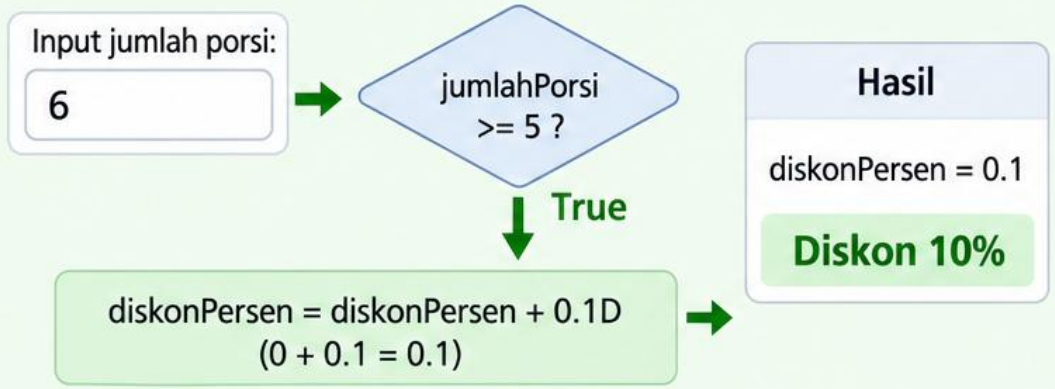
If jumlahPorsi >= 5 Then
    diskonPersen = diskonPersen + 0.1D
End If
```

Penjelasan Singkat

1. **diskonPersen** dibuat dengan nilai awal 0 (belum ada diskon).
2. Cek apakah jumlahPorsi ≥ 5.
3. Jika **True**, diskon ditambah 0.1 (10%).
4. Jika **False**, blok If dilewati dan diskon tetap 0%.

Contoh 1: jumlahPorsi ≥ 5

Mendapat diskon 10%



Contoh 2: jumlahPorsi < 5

Tidak mendapat diskon



Kesimpulan

Kalau beli minimal 5 porsi, dapat diskon 10%.
Kalau kurang dari 5, tidak ada diskon.

Kasus Diskon — Percabangan 10%

Kondisi pertama bergantung pada jumlah porsi.

```
Dim diskonPersen As Decimal = 0D

If jumlahPorsi >= 5 Then
    diskonPersen = diskonPersen +
0.1D
End If
```

Makna

Jika jumlah porsi minimal 5, persentase diskon bertambah 10%. Jika tidak, nilainya tetap 0.

Kasus Diskon — Tambahan Member 5%

Gunakan And karena kedua syarat harus benar.

```
If isMember And subtotal >= 150000D
Then
    diskonPersen = diskonPersen +
0.05D
End If
```

Contoh

Member + subtotal Rp175.000 → dapat tambahan 5%.

Bukan member + subtotal Rp175.000 → tidak dapat tambahan 5%.

Kasus Diskon — Membatasi Maksimal 15%

Percabangan juga dapat digunakan untuk menetapkan batas.

```
If diskonPersen > 0.15D Then
    diskonPersen = 0.15D
End If
```

Tujuan

Apa pun kombinasi aturan diskon, program memastikan total persentase diskon tidak melebihi 15%.

Kasus Diskon — Menentukan Ongkir

Ongkir hanya muncul jika total setelah diskon masih di bawah Rp200.000.

```
If totalSetelahDiskon < 200000D
Then
    ongkir = 10000D
Else
    ongkir = 0D
End If
```

Dua hasil

True → ongkir Rp10.000
False → ongkir Rp0

Studi Kasus: Diskon Pemesanan Makanan

Ketentuan

1. Harga makanan = Rp35.000/porsi.
2. Minimal 5 porsi → diskon 10%.
3. Member DAN subtotal ≥ Rp150.000 → tambahan diskon 5%.
4. Total diskon maksimal 15%.
5. Setelah diskon dikenakan biaya layanan 5%.
6. Jika total setelah diskon < Rp200.000 → ongkir Rp10.000.

Kode Program (VB.NET)

```
Private Sub btnHitung_Click(sender As Object, e As EventArgs)
    Handles btnHitung.Click

    ' Deklarasi variabel
    Dim jumlahPorsi As Integer
    Dim harga As Decimal = 35000D
    Dim diskonPersen As Decimal = 0D
    Dim subtotal As Decimal
    Dim totalSetelahDiskon As Decimal
    Dim biayaLayanan As Decimal
    Dim ongkir As Decimal = 0D
    Dim totalBayar As Decimal

    ' Validasi input
    If Not Integer.TryParse(txtJumlah.Text, jumlahPorsi) Then
        MessageBox.Show("Masukkan jumlah porsi yang benar.")
        Exit Sub
    End If

    ' Hitung subtotal
    subtotal = jumlahPorsi * harga

    ' Diskon 10% jika minimal 5 porsi
    If jumlahPorsi >= 5 Then
        diskonPersen = diskonPersen + 0.1D
    End If

    ' Tambahan diskon 5% jika member DAN subtotal >= 150.000
    If chkMember.Checked And subtotal >= 150000D Then
        diskonPersen = diskonPersen + 0.05D
    End If

    ' Batas maksimal diskon 15%
    If diskonPersen > 0.15D Then
        diskonPersen = 0.15D
    End If

    ' Hitung total setelah diskon
    totalSetelahDiskon = subtotal -
        (subtotal * diskonPersen)

    ' Biaya layanan 5%
    biayaLayanan = totalSetelahDiskon * 0.05D

    ' Ongkir Rp10.000 jika total setelah
    ' diskon < 200.000
    If totalSetelahDiskon < 200000D Then
        ongkir = 10000D
    End If

    ' Total pembayaran
    totalBayar = totalSetelahDiskon +
        biayaLayanan + ongkir

    ' Tampilkan hasil
    lblSubtotal.Text = "Subtotal: Rp" &
        subtotal.ToString("N0")
    lblDiskon.Text = "Diskon: " &
        (diskonPersen * 100).ToString("0") & "%"
    lblSetelahDiskon.Text = "Total Setelah
    Diskon: Rp" & totalSetelahDiskon.ToString("N0")
    lblLayanan.Text = "Biaya Layanan (5%): Rp" &
        biayaLayanan.ToString("N0")
    lblOngkir.Text = "Ongkir: Rp" &
        ongkir.ToString("N0")
    lblTotal.Text = "Total Bayar: Rp" &
        totalBayar.ToString("N0")

End Sub
```

Contoh Tampilan Form (GUI)



Contoh Perhitungan (5 porsi, member)

1. **Subtotal**
   5 × Rp35.000 = Rp175.000
2. **Diskon**
   - Minimal 5 porsi → 10%
   - Member DAN subtotal ≥ Rp150.000 → 5%

   Total diskon = 10% + 5% = 15% (maksimal 15%)
   Nilai diskon = 15% × Rp175.000 = Rp26.250
3. **Total setelah diskon**
   Rp175.000 − Rp26.250 = Rp148.750
4. **Biaya layanan 5%**
   5% × Rp148.750 = Rp7.437,50 (≈ Rp7.438)
5. **Ongkir**
   Rp148.750 < Rp200.000 → ongkir = Rp10.000
6. **Total bayar**
   Rp148.750 + Rp7.438 + Rp10.000 = **Rp166.188**

Kesimpulan

- Program menggunakan operator aritmatika, perbandingan, dan logika (And).
- Diskon dihitung bertahap dan dibatasi maksimal 15%.
- Biaya layanan dikenakan setelah diskon.
- Ongkir hanya ditambahkan jika total setelah diskon < Rp200.000.

LATIHAN

Simulasi Kasus 1

Pesanan 4 porsi, bukan member.



Diskusikan

Berapa biaya layanan 5% dan total bayar akhirnya?

LATIHAN

Simulasi Kasus 2

Pesanan 5 porsi dan member.



Pertanyaan

Mengapa diskon menjadi 15%? Berapa biaya layanan dan total bayar?

LATIHAN

Mini Challenge — Aplikasi Tarif Parkir

Rancang logika sebelum menulis kode.

Input: lama parkir (jam).

1–2 jam → Rp5.000.

3–5 jam → Rp10.000.

> 5 jam → Rp15.000.

Jika member, potongan Rp2.000.

Tampilkan tarif akhir pada Label.

Pertanyaan pemantik

Percabangan apa yang paling cocok?

Di mana operator And diperlukan?

Bagaimana memastikan lama parkir tidak negatif?

Praktik Mandiri

Bangun satu aplikasi sederhana menggunakan percabangan.

Pilihan kasus

1. Tarif laundry
2. Belanja Toko
3. Penyewaan Mobil
4. Pemesanan Tiket Bioskop
5. Tarif Hotel

Wajib ada

Minimal 2 input
Minimal 1 Button
If/ElseIf/Else
Validasi input
Output pada Label atau MessageBox